Sales Trend Analysis Using Aggregations

1. Objective

To analyze monthly revenue and order volume using SQL aggregation functions.

---

2. Tools Used

- MySQL Workbench
- SQL (GROUP BY, SUM, COUNT functions)

---

3. Dataset Description

The dataset `online_sales` contains the following columns:

- `order_id` – Unique ID for each order
- `order_date` – Date of the order
- `amount` – Total amount of the order
- `product_id` – Product identifier

---

4. SQL Script

```
CREATE DATABASE sale_db;

USE sale_db;

SHOW DATABASES;

CREATE TABLE online_sales(order_id INT, order_date DATE, amount DECIMAL(10,2), product_id INT);

INSERT INTO online_sales VALUES (1,'2024-01-10',500.00,101),(2,'2024-01-15',300.00,102),(3,'2024-02-05',700.00,103),(4,'2024-02-20',200.00,101),(5,'2024-03-12',900.00,104);

SELECT YEAR(order_date) AS year, MONTH(order_date) AS month, SUM(amount) AS total_revenue, COUNT(DISTINCT order_id) AS total_orders FROM online_sales GROUP BY YEAR(order_date), MONTH(order_date) ORDER BY year, month;

SELECT YEAR(order_date) AS year, MONTHA(order_date) AS month, SUM(amount) AS total_revenue, COUNT(DISTINCT order_id) AS total_orders FROM online_sales WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31' GROUP BY YEAR(order_date), MONTH(order_date) ORDER BY year, month;
```

5. Output

year	month	total_revenue	total_orders
2024	1	800.00	2
2024	2	900.00	2
2024	3	900.00	1

6. Observation

- Revenue increased from January to February.
- February and March recorded the highest revenue (900).
- Order volume is highest in January and February.
- March shows high revenue but fewer orders, indicating higher-value transactions.

---

7. Conclusion

The analysis demonstrates how aggregation functions like `SUM()` and `COUNT()` can be used to evaluate monthly sales performance. Grouping data by year and month helps in identifying trends and patterns, which can support better business decision-making.